

מעבדה בבינה מלאכותית

דו"ח תרגיל בית 1

חלק א: rush hour

1. הלוח הוא רשת של 6 על 6, שכל רכב מתואר על ידי ID, עם כיוון ואורך. המצב (state) מיוצג בתור dataclass קפוא שמכיל frozenset של vehicle dataclasses

```
2. # src/part_a/state.py
@dataclass(frozen=True)
class Vehicle:
    id: str
    is_horizontal: bool
    length: int
    row: int
    col: int

@dataclass(frozen=True)
class State:
    vehicles: FrozenSet[Vehicle]
    grid_size: int = 6

    def get_vehicle(self, vehicle_id: str) -> Optional[Vehicle]:
        for v in self.vehicles:
            if v.id == vehicle_id:
                return v
        return None
```

השימוש בfrozenset הוא חשוב, כי הוא immutable ולא תלוי בסדר הרכבים. לכן ניתן לבצע hashing על הסטייט ולהכניס ישר לset של מצבים ואז ניתן לבצע בדיקות ב $O(1)$ באמצעות setn. המטרה היא להוציא את רכב X וניתן להזיז כל רכב תא לכל כיוון שמותר לו.

2. אלגוריתמי חיפוש:

a. BFS – האלגוריתם הראשון שבוצע איתו החיפוש הוא אלגוריתם "נאיבי" יותר בלי יורסטיקות כדי לקבל את הבסיס של חיפוש כאשר הוא לא מודע.

```
# src/part_a/search.py
frontier = deque([node]) # FIFO O(1) append ו-popleft
explored = {node.state} # set לבדיקת O(1)

while frontier:
    current_node = frontier.popleft()
    for action in problem.get_actions(current_node.state):
        child_state = problem.get_result(current_node.state, action)
        if child_state not in explored:
            if problem.is_goal(child_state):
                return child_node
```

```
explored.add(child_state)
frontier.append(child_node)
```

BFS רץ בזמן לינארי באופן כמעט תמידי לכן frontier יכול להגיע לעשרות אלפי צמתים.

b. A^* - שימוש ב A^* כמו שלמדנו, A^* משתמש בתור עדיפיות עם ערימת מינימום שממויין לפי $f(n)=g(n)+h(n)$ כאשר g היא עלות המסלול ו h זה האומדן היורסיטי ולכן החיפוש הוא על node שנראים הכי עדיפים

```
# src/part_a/search.py
frontier = []
heapq.heappush(frontier, initial_node)
explored = {initial_node.state: initial_node.path_cost}

while frontier:
    current_node = heapq.heappop(frontier) # O(Log n)
    if problem.is_goal(current_node.state):
        return current_node
    for action in actions:
        child_cost = current_node.path_cost + 1
        if child_state not in explored or child_cost < explored[child_state]:
            explored[child_state] = child_cost
            heapq.heappush(frontier, child_node)
```

יוריסטיקות:

H1 ספירת חוסמים:

$$h1(s) = |\{X \text{ רכבים אנכיים בנתיב היציאה של } s\}| + 1$$

הפלוס 1 הוא בשביל X עצמו, כל רכב שחוסם את X חייב לזוז לפחות פעם אחת ו X עצמו חייב לזוז לפחות פעם אחת, $h1$ לעולם לא יהיה בערכת יתר ולכן הוא אדמיסיבילי.

H2 חוסמים + חוסמים תקועים

$$h2(s) = h1(s) + |\{\text{חוסמים שאינם יכולים לזוז בשני הכיוונים}\}|$$

חוסם "תקוע" הוא רכב אנכי שגם מעליו וגם מתחתיו חוסם (גבול הלוח או רכב אחר). רכב כזה דורש לפחות עוד מהלך אחד כדי לפנות לו מקום לפני שהוא יכול לזוז. לכן הוספת 1 לכל חוסם תקוע היא עדיין חסם תחתון תקף

```
# src/part_a/problem.py
h2 = h1
for car_id in blocking_cars:
    car = state.get_vehicle(car_id)
    if not car.is_horizontal:
        can_move_up = car.row > 0 and grid[car.row - 1][car.col] is None
        can_move_down = (car.row + car.length < state.grid_size
                           and grid[car.row + car.length][car.col] is None)
```

```

if not can_move_up and not can_move_down:
    h2 += 1

```

אדמיסיביליות: שתי ההיוריסטיקות סופרות רק מהלכים שחייבים לקרות לכן אין הערכת יתר.

עקביות (מונטוניות): כל פעולה עולה 1, בכל צעד h יכול לרדת לכל היותר ב 1

לכן, $h(n) \leq 1 + h(n')$, כלומר $f(n)$ לא יורד.

מבני נתונים:

שימוש	סיבוכיות	פעולה	מבנה
BFS תור	$O(1)$	append / popleft	deque
A^* תור עדיפויות של	$O(\log n)$	heappush / heappop	heapq
מצבים שבוקרו	ממוצע $O(1)$	in, add	set / dict

ה frozenset - בתוך State מאפשר hashing של כל מצב ב $|(\text{vehicles})|$ - מאחר שיש לכל היותר 15 רכבים בלוח, זה למעשה כמו $O(1)$.

תוצאות:

הריצות של שלושת האלגוריתמים על כל ה 40 חידות נפתרו בהצלחה על ידי כל האלגוריתמים

BFS

Problem	Heuristic	N	d/N	Success	Time(ms)	EBF	avg	Min	Avg	Max
1	h1	1012	0.008	Y	225.37	3.049	N/A	0	5.58	8
2	h1	1547	0.005	Y	431.63	3.206	N/A	0	6.24	8
3	h1	661	0.021	Y	129.63	1.764	N/A	0	9.06	14
4	h1	286	0.032	Y	55.51	2.26	N/A	0	7.81	9
5	h1	1740	0.005	Y	502.58	2.85	N/A	0	7.09	9
6	h1	1450	0.006	Y	412.24	2.772	N/A	0	6.39	9
7	h1	3368	0.004	Y	790.67	2.158	N/A	0	9.51	13

Problem	Heuristic	N	d/N	Success	Time(ms)	EBF	Hang	Min	Avg	Max
8	h1	948	0.013	Y	185.88	1.958	N/A	0	7.55	12
9	h1	502	0.024	Y	117.39	1.885	N/A	0	9.55	12
10	h1	1630	0.01	Y	354.44	1.658	N/A	0	11.12	17
11	h1	783	0.032	Y	120.51	1.345	N/A	0	14.81	25
12	h1	1024	0.017	Y	178.5	1.615	N/A	0	12.49	17
13	h1	6752	0.002	Y	1544.38	1.901	N/A	0	12.26	16
14	h1	7502	0.002	Y	1870.63	1.867	N/A	0	14.61	17
15	h1	521	0.044	Y	64.35	1.336	N/A	0	9.22	23
16	h1	2094	0.01	Y	322.12	1.518	N/A	0	10.76	21
17	h1	2056	0.012	Y	335.24	1.437	N/A	0	14.89	24
18	h1	1497	0.017	Y	189.73	1.391	N/A	0	17.9	25
19	h1	471	0.047	Y	50.72	1.366	N/A	0	8.44	22
20	h1	1001	0.01	Y	138.84	2.342	N/A	0	8.19	10
21	h1	247	0.085	Y	22.85	1.332	N/A	0	11.7	21
22	h1	3053	0.009	Y	474.93	1.411	N/A	0	21.54	26
23	h1	1953	0.015	Y	266.33	1.332	N/A	0	24.29	29
24	h1	4175	0.006	Y	663.22	1.464	N/A	0	10.21	25
25	h1	8027	0.003	Y	1254.47	1.456	N/A	0	19.8	27
26	h1	4675	0.006	Y	635.64	1.395	N/A	0	18.96	28
27	h1	2558	0.011	Y	276.56	1.355	N/A	0	20.65	28
28	h1	1674	0.018	Y	202.27	1.311	N/A	0	24.89	30
29	h1	4301	0.007	Y	555.01	1.342	N/A	0	16.56	31

Problem	Heuristic	N	d/N	Success	Time(ms)	EBF	Havg	Min	Avg	Max
30	h1	1122	0.029	Y	112.98	1.266	N/A	0	23.39	32
31	h1	3841	0.01	Y	565.29	1.271	N/A	0	22.85	37
32	h1	554	0.067	Y	52.82	1.183	N/A	0	17.84	37
33	h1	3886	0.01	Y	580.31	1.246	N/A	0	29.26	40
34	h1	4295	0.01	Y	615.71	1.224	N/A	0	26.59	43
35	h1	3852	0.011	Y	519.99	1.226	N/A	0	21.34	43
36	h1	2268	0.019	Y	625.49	1.202	N/A	0	28.85	44
37	h1	1934	0.024	Y	551.23	1.178	N/A	0	35.37	47
38	h1	3521	0.014	Y	684.91	1.192	N/A	0	36.69	48
39	h1	3539	0.014	Y	506.37	1.178	N/A	0	24.39	50
40	h1	2810	0.018	Y	444.44	1.17	N/A	0	36.91	51

A* on h1

Problem	Heuristic	N	d/N	Success	Time(ms)	EBF	Havg	Min	Avg	Max
1	h1	685	0.012	Y	113.5	2.193	2.804	0	4.51	8
2	h1	527	0.015	Y	83.49	2.238	3.331	0	4.79	8
3	h1	464	0.03	Y	47.56	1.464	2.634	0	7.79	14
4	h1	138	0.065	Y	15.2	1.71	2.478	0	6.64	9
5	h1	697	0.013	Y	110.15	2.078	3.447	0	5.61	9
6	h1	567	0.016	Y	81.06	2.009	3.918	0	4.71	9
7	h1	2189	0.006	Y	401.39	1.733	2.917	0	7.69	13
8	h1	871	0.014	Y	111.2	1.65	3.724	0	6.73	12
9	h1	348	0.035	Y	45.27	1.556	2.454	0	8.72	12

Problem	Heuristic	N	d/N	Success	Time(ms)	EBF	Havg	Min	Avg	Max
10	h1	1491	0.011	Y	222.72	1.444	2.175	0	10.29	17
11	h1	709	0.035	Y	71.84	1.22	2.677	0	13.54	25
12	h1	635	0.027	Y	73.74	1.377	2.921	0	10.43	17
13	h1	3893	0.004	Y	669.02	1.605	3.406	0	10.26	16
14	h1	3750	0.004	Y	614.43	1.559	2.778	0	12.36	17
15	h1	523	0.044	Y	53.3	1.221	3.799	0	8.63	23
16	h1	1987	0.011	Y	264.37	1.357	2.814	0	10.58	21
17	h1	1789	0.013	Y	282.07	1.289	3.743	0	13.82	24
18	h1	1463	0.017	Y	165.99	1.264	2.244	0	16.94	25
19	h1	471	0.047	Y	44.95	1.228	2.236	0	8.97	22
20	h1	313	0.032	Y	39.55	1.779	3.043	0	6.55	10
21	h1	238	0.088	Y	18.62	1.199	2.132	0	10.65	21
22	h1	2763	0.009	Y	411.22	1.294	2.707	0	20.47	26
23	h1	1483	0.02	Y	183.31	1.22	3.342	0	22.25	29
24	h1	4028	0.006	Y	621.23	1.319	2.23	0	9.9	25
25	h1	6757	0.004	Y	996.94	1.322	3.451	0	18.17	27
26	h1	4383	0.006	Y	581.3	1.283	3.275	0	17.07	28
27	h1	2527	0.011	Y	305.37	1.254	3.369	0	19.32	28
28	h1	1401	0.021	Y	203.18	1.208	3.374	0	23.39	30
29	h1	5340	0.006	Y	696.69	1.255	4.005	0	15.83	31
30	h1	1048	0.03	Y	114.1	1.178	2.39	0	21.88	32

Problem	Heuristic	N	d/N	Success	Time(ms)	EBF	Havg	Min	Avg	Max
31	h1	3870	0.01	Y	455.67	1.19	3.256	0	22.0	37
32	h1	530	0.07	Y	45.14	1.118	3.274	0	16.46	37
33	h1	2665	0.015	Y	373.03	1.168	4.037	0	23.97	40
34	h1	4468	0.01	Y	598.15	1.166	3.856	0	24.73	43
35	h1	4427	0.01	Y	532.95	1.159	3.888	0	20.43	43
36	h1	2036	0.022	Y	290.3	1.139	2.164	0	27.74	44
37	h1	1978	0.024	Y	261.79	1.125	3.723	0	33.55	47
38	h1	3273	0.015	Y	389.77	1.137	3.226	0	34.86	48
39	h1	3683	0.014	Y	386.17	1.132	3.01	0	24.34	50
40	h1	2555	0.02	Y	316.87	1.116	2.801	0	34.54	51

A* on h2

Problem	Heuristic	N	d/N	Success	Time(ms)	EBF	Havg	Min	Avg	Max
1	h2	649	0.012	Y	111.41	2.202	2.973	0	4.55	8
2	h2	297	0.027	Y	51.89	2.112	3.63	0	4.41	8
3	h2	435	0.032	Y	54.48	1.459	3.063	0	7.69	14
4	h2	73	0.123	Y	9.2	1.619	3.191	0	6.12	9
5	h2	424	0.021	Y	79.37	2.015	4.237	0	5.31	9
6	h2	407	0.022	Y	70.53	1.964	4.806	0	4.44	9
7	h2	2149	0.006	Y	316.6	1.73	3.282	0	7.52	13
8	h2	613	0.02	Y	89.37	1.632	4.6	0	6.42	12
9	h2	318	0.038	Y	45.94	1.553	2.83	0	8.59	12

Problem	Heuristic	N	d/N	Success	Time(ms)	EBF	Havg	Min	Avg	Max
10	h2	1421	0.012	Y	222.27	1.438	2.707	0	9.9	17
11	h2	671	0.037	Y	81.59	1.226	3.621	0	13.49	25
12	h2	555	0.031	Y	69.55	1.355	3.803	0	9.51	17
13	h2	3639	0.004	Y	843.11	1.61	4.461	0	10.3	16
14	h2	3256	0.005	Y	648.89	1.545	3.799	0	11.98	17
15	h2	536	0.043	Y	70.35	1.219	5.221	0	8.58	23
16	h2	1841	0.011	Y	297.76	1.348	4.257	0	9.39	21
17	h2	1858	0.013	Y	305.74	1.28	5.348	0	14.33	25
18	h2	1130	0.022	Y	170.22	1.257	3.284	0	16.02	25
19	h2	457	0.048	Y	54.02	1.232	2.864	0	8.67	22
20	h2	256	0.039	Y	42.29	1.77	3.583	0	6.66	10
21	h2	222	0.095	Y	23.27	1.193	3.042	0	10.76	21
22	h2	2381	0.011	Y	386.46	1.29	3.317	0	20.11	26
23	h2	1265	0.023	Y	195.27	1.216	4.32	0	21.49	29
24	h2	3983	0.006	Y	1873.46	1.322	2.942	0	9.46	25
25	h2	6396	0.004	Y	2747.48	1.32	4.592	0	17.41	27
26	h2	4343	0.006	Y	1348.06	1.276	3.753	0	16.77	28
27	h2	2441	0.011	Y	654.27	1.253	4.238	0	18.97	28
28	h2	971	0.031	Y	322.42	1.199	4.433	0	22.35	30
29	h2	5206	0.006	Y	1643.02	1.252	5.554	0	15.98	31
30	h2	964	0.033	Y	250.39	1.171	3.088	0	21.46	32
31	h2	4005	0.009	Y	1111.99	1.191	4.206	0	22.1	37

Problem	Heuristic	N	d/N	Success	Time(ms)	EBF	Havg	Min	Avg	Max
32	h2	514	0.072	Y	113.63	1.117	4.183	0	16.45	37
33	h2	2392	0.017	Y	658.28	1.161	5.515	0	21.56	40
34	h2	4605	0.009	Y	1384.27	1.165	4.915	0	24.65	43
35	h2	4504	0.009	Y	1086.89	1.162	5.211	0	20.61	43
36	h2	1876	0.024	Y	444.44	1.133	2.821	0	26.05	44
37	h2	2171	0.022	Y	699.04	1.127	4.897	0	34.16	47
38	h2	3287	0.015	Y	1117.65	1.132	4.253	0	34.45	48
39	h2	3882	0.013	Y	1251.64	1.132	4.202	0	24.18	50
40	h2	2609	0.019	Y	947.91	1.122	3.809	0	34.56	51

סיכום תוצאות:

אלגוריתם	N ממוצע	EBF	d/N	Havg	זמן ממוצע (ms)
BFS	2,478	1.635	0.0177	-	441
A* h1	2,074	1.399	0.0215	3.08	282
A* h2	1,975	1.388	0.0251	3.97	547

אפשר לשים לב שהEBF יורד ככל שהחידות קשות יותר, הוא בBFS יורד ל1.21 כי עץ החיפוש עצמו כבר מתחיל להיות מוגבל יותר, A* עם h2 מייצר פחות נודס לעומת h1 (1975 מול 2074 בממוצע) אבל לוקח יותר זמן לפתרון ממוצע (547ms מול 282ms) כי כל הערכת נוד יקרה יותר, ואז הבדיקה של התקיעות של כל חוסם. לכן עבור הבעיה הזאת A* עם h1 הוא האיזון המועדף.

קובץ פתרונות

כל הרצה לחפש את הפתרונות מוציא את הפלט לresults/solutions_{algo}_{heuristic}.txt וכל שורה מייצגת חידה אחת עם הפתרון במחרוזת.

```
$ python main.py part_a --board_file data/rh.txt --puzzle_index 1 --algorithm astar --heuristic h1 --verbose
```

```
Loading Rush Hour board from: data/rh.txt (Puzzle Index: 1)
Running A* Search...
```

```
--- Metrics ---
```

Time(ms): 65.04
N: 664
d/N: 0.012
EBF: 2.211
Havg: 2.812
Min Depth: 0
Avg Depth: 4.59
Max Depth: 8

Solution found! Path length: 8
Path: CL3 OD3 AR3 PU1 BU1 RL2 QD2 XR3

--- Step-by-Step Solution ---
Step 1: Move C L3
Step 2: Move O D3
Step 3: Move A R3
Step 4: Move P U1
Step 5: Move B U1
Step 6: Move R L2
Step 7: Move Q D2
Step 8: Move X R3

השוואה מול הפתרונות הנתונים:

קובץ RH מכיל את הפתרונות ולכן ניתן לבדוק איזה פתרון עדיף, לפי התוצאות של האלגוריתמים בחידות 1 עד 10, 12, 13, 20 הפתרונות שקיבלתי תואמת בדיוק את המסלול האופטימלי,

בשאר הפתרונות לא בטוח שהפתרון המלא הוא הנכון כי יצא פתרון לשאלה 40 שכלל 51 מהלכים, יכול להיות שזה טעות של האלגוריתם או של הקובץ פתרונות.

מחולל חידות חדשות:

המימוש של מחולל החידות הוא `src/part_a/generator.py`

צריך לעמוד ב3 מאפיינים כדי שיהיה תקין:

1. בין 1 ל3 רכבים אנכיים בנתיב היציאה של X
2. לפחות 12 תאים תפוסים
3. לפחות חוסם אחד תקוע.

הוספת הקוד של המחולל

חלק ב:

כיסוי קבוצות ממושקל עם אלגוריתם גנטי

1. הגדרת הבעיה: בבית set cover נתונים לנו m אלמנטים וח קבוצות עם עלויות, המטרה שלנו היא למצוא תת קבוצה בעלת עלות מינמלית שמכסה את כל האלמנטים.

בעיות שבוצעו:

אופטימום ידוע	ח (קבוצות)	m (אלמנטים)	מערך נתונים
429	1,000	200	scp41
512	1,000	200	scp42
516	1,000	200	scp43
494	1,000	200	scp44
253	2,000	200	scp51
302	2,000	200	scp52
226	2,000	200	scp53
253	3,000	300	scpa1
252	3,000	300	scpa2
232	3,000	300	scpa3

2. קוד הכרומוזום:

כל כרומוזום הוא מערך בינארי באורך n אם $i=1$ אז הוא נבחר, אחרת לא נבחר

```
# src/part_b/chromosome.py
@dataclass
class Chromosome:
    genes: List[int] # בינארי, אורך n
    fitness: float = float('inf')
    is_valid: bool = False
    cost: float = float('inf')
```

3. תיקון פיטנס וטרים:

4. כרומוזום אקראי כמעט תמיד יפספס אלמנטים מסוימים, ובמקום להעניש פתרונות לא

חוקיים נבחר לקחת אותך תוך כדי עם `evaluate()`. התוצאה היא שהאוכלוסייה תמיד

מורכבת מפתרונות חוקיים והפיטנס הוא פשוט העלות הכוללת.

שלב התיקון: יש זיהוי של אלמנטים לא מכוסים, אז מוסיפים בצורה גרידית את הקבוצה הלא נבחרת עם יחס עלות כיסוי חדש שיהיה הטוב ביותר עד שהכל מכוסה. שלב הטרים: עוברים על הקבוצות הנבחרות לפי סדר עלות יורד, ואז מסירים אם על האלמטים שלה מכוסים על ידי קבוצות אחרות כבר

```
# src/part_b/chromosome.py evaluate()
```

```
# שלב 1: תיקון הוסף קבוצות עד שהכל מכוסה
```

```
uncovered = [e for e, count in enumerate(covered_counts) if count == 0]
while uncovered:
    best_set = -1
    best_ratio = float('inf')
    for s_idx in range(problem.n):
```

```

    if self.genes[s_idx] == 1:
        continue
    new_covered = sum(1 for e in problem.sets_to_elements[s_idx]
                      if covered_counts[e] == 0)
    if new_covered > 0:
        ratio = problem.costs[s_idx] / new_covered
        if ratio < best_ratio:
            best_ratio = ratio
            best_set = s_idx
    if best_set != -1:
        self.genes[best_set] = 1
        for e in problem.sets_to_elements[best_set]:
            covered_counts[e] += 1
    uncovered = [e for e in uncovered if covered_counts[e] == 0]

# 2 שלב: Trim (היקרות קודם) הסר קבוצות מיותרות
selected_sets.sort(key=lambda idx: problem.costs[idx], reverse=True)
for s_idx in selected_sets:
    is_redundant = all(covered_counts[e] >= 2
                      for e in problem.sets_to_elements[s_idx])
    if is_redundant:
        self.genes[s_idx] = 0
        for e in problem.sets_to_elements[s_idx]:
            covered_counts[e] -= 1

# שלב 3: חישוב עלות סופית
self.cost = sum(problem.costs[i] for i, v in enumerate(self.genes) if v == 1)
self.is_valid = True
self.fitness = self.cost # תמיד חוקי, אין עונש

```

5. אתחול אוכלוסייה: אוכלוסייה של 300 כרומוזומים, כל ג'ן מוגרל ל1 בהסתברות של $n/2$, שלב התיקון ממלא כל כרומוזום בלפתרון חוקים. התחלה שהיא ספארסית מאלצת את מנגנון התיקון לבחורות בצורה מגוונת את הבחירה הראשונית ולכן יש אוכלוסייה איכותית יחסית להתחלה.
6. סלקציה: סלקציה בטורניר עם $k=3$ ממאגר הבחירה, לכן רק 50% הטובים ביותר של האוכלוסייה (שגם ממוינת לפי פיטנס) ואז מושכים את 3 מתמודדים אקראיים, כך שהמנצח יהיה בעל הפיטנס הנמוך ביותר. ההגבלה ל50% מספקת לחץ סלקציה נוסף מבלי להוציא לגמרי חצי מהאוכלוסייה.

```

# src/part_b/ga_engine.py
mating_pool = self.population[:mating_pool_size] # top 50%
contenders = self.random.sample(mating_pool, k=3)
return min(contenders, key=lambda c: c.fitness)

```

7. הכלאה: שימוש בuniform crossover בשיעור של 0.8 לכל מיקום בג', מגרילים 50/50 מאיזה הורה לקחת. לכן הילדים שהם פסיפס של שני ההורים ולאו דווקא חתך בנקודה אחת.

```
# src/part_b/ga_engine.py
def crossover(self, p1, p2):
    if self.random.random() < self.crossover_rate:
        c1_genes, c2_genes = [], []
        for g1, g2 in zip(p1.genes, p2.genes):
            if self.random.random() < 0.5:
                c1_genes.append(g1); c2_genes.append(g2)
            else:
                c1_genes.append(g2); c2_genes.append(g1)
        return Chromosome(c1_genes), Chromosome(c2_genes)
    return Chromosome(list(p1.genes)), Chromosome(list(p2.genes))
```

8. מוטציה: שימוש בbit flip בשיעור של 0.05 לג', כלומר על ג'ן מתהפך בהסתברות של 5% באופן עצמאי. בגלל שמנגנון התיקון מתקן תוצאות לא חוקיות אז מוטציות יכולה בחופשיות להוריד קבוצות, אבל אם היו נחוצות הן יתווספו חזרה בשלב התיקון.
9. אליטיזם: 10% מהאוכלוסייה הן הטובים ביותר שעוברים מדור לדור הבא ללא שינוי, לכן זה מבטיח שפתרונות טובים לא אובדים בשלבים של הכלאה או מוטציה.
10. עצירה מוקדמת יש שני תנאים לעצירה לפני 200 דורות:

a. Stagnation: אם הפיטנס הטוב ביותר לא השתפר במשך 30 דורות רצופים

b. Entropy convergence: אם האנטרופיה הממוצעת של האוכלוסייה ירדה מתחת ל-5%

11. מדדי גיוון: יש מדידה של 3 מדדי גיוון בכל דור

א. אנטרופיה שאנונית אם היא קרובה ל1 או קרובה ל0

ב. מרחק המינג ממוצע

ג. לחץ סלקציה, ככל שגבוה יותר כך על הסלקציה להיות אגריסיבית יותר

```
# src/part_b/ga_engine.py record_metrics()

matrix = np.array([c.genes for c in self.population]) # (pop_size, n)
p_1 = matrix.mean(axis=0) # תדירות ה-1 בכל מיקום
p_0 = 1 - p_1

# אנטרופיה שאנונית ממוצע על פני כל המיקומים
p_1_clipped = np.clip(p_1, 1e-9, 1 - 1e-9)
p_0_clipped = np.clip(p_0, 1e-9, 1 - 1e-9)
entropy = float(
    -np.sum(p_1_clipped * np.log2(p_1_clipped)
    + p_0_clipped * np.log2(p_0_clipped)) / self.problem.n
)

# ממוצע בין-זוגי צפוי Hamming מרחק
hamming_distance = float(np.sum(2 * p_1 * p_0))
```

```
# (גבוה = לחץ גבוה יותר) avg_fitness / best_fitness : לחץ סלקציה
selection_pressure = float(avg / best) if best > 0 else 1.0
```

12. בייסליין גרידי: בכל שלב הוא בוחר את הקבוצה עם יחס עלות אלמנטים חדשים הנמוך ביותר. רץ ב60ms ואז משמש כקו בסיס, האלגוריתם הגנטי חייב לנצח אותו

```
$ python main.py part_b --dataset data/setcover/scp41.txt --seed 4
2
```

```
Loading Set Cover Dataset: data/setcover/scp41.txt
Loaded: m=200 elements, n=1000 sets
```

```
--- Greedy Baseline ---
```

```
Cost: 463.0
```

```
Time: 56.48 ms
```

```
--- Genetic Algorithm ---
```

```
Best Configuration Found:
```

```
Valid: True
```

```
Fitness (Cost+Penalty): 433.0
```

```
Actual Cost: 433.0
```

```
Algorithm Time: 16929.44 ms
```

13. השוואת 3 קונפיגורציות (5,10 וסידס שונים): הבדיקה על שלוש קונפיגורציות של אופרטורים כדי להבין מה מניע את הביצועים

שיעור מוטציה	שיעור הכלאה	קונפיגורציה
0.0	0.8	Crossover Only
0.05	0.0	Mutation Only
0.05	0.8	Both

כל קונפיגורציה רצה 4 פעמים עם סידס שונים (42,123,999,20267) על כל 10 מערכי הנתונים

דורות ממוצעים	פער מאופטימלי (%)	עלות ממוצעת	קונפיגורציה	מערך נתונים
—	+7.93%	463.00	Greedy	scp41.txt
1	+1.17%	434.00	Crossover Only	scp41.txt
1	+0.93%	433.00	Mutation Only	scp41.txt
1	+0.93%	433.00	Both	scp41.txt
—	+13.67%	582.00	Greedy	scp42.txt
1	+1.81%	521.25	Crossover Only	scp42.txt
1	+1.85%	521.50	Mutation Only	scp42.txt
1	+1.90%	521.75	Both	scp42.txt

מערך נתונים	קונפיגורציה	עלות ממוצעת	פער מאופטימלי (%)	דורות ממוצעים
scp43.txt	Greedy	598.00	+15.89%	—
scp43.txt	Crossover Only	529.75	+2.66%	1
scp43.txt	Mutation Only	530.25	+2.76%	1
scp43.txt	Both	531.75	+3.05%	1
scp44.txt	Greedy	548.00	+10.93%	—
scp44.txt	Crossover Only	502.25	+1.67%	1
scp44.txt	Mutation Only	501.00	+1.42%	1
scp44.txt	Both	502.00	+1.62%	1
scp51.txt	Greedy	289.00	+14.23%	—
scp51.txt	Crossover Only	268.50	+6.12%	1
scp51.txt	Mutation Only	268.50	+6.12%	1
scp51.txt	Both	267.75	+5.83%	1
scp52.txt	Greedy	348.00	+15.23%	—
scp52.txt	Crossover Only	321.50	+6.46%	1
scp52.txt	Mutation Only	323.00	+6.95%	1
scp52.txt	Both	323.00	+6.95%	1
scp53.txt	Greedy	246.00	+8.85%	—
scp53.txt	Crossover Only	231.00	+2.21%	1
scp53.txt	Mutation Only	230.00	+1.77%	1
scp53.txt	Both	230.00	+1.77%	1
scpa1.txt	Greedy	288.00	+13.83%	—
scpa1.txt	Crossover Only	258.25	+2.08%	1
scpa1.txt	Mutation Only	258.75	+2.27%	1
scpa1.txt	Both	258.75	+2.27%	1
scpa2.txt	Greedy	284.00	+12.70%	—
scpa2.txt	Crossover Only	261.75	+3.87%	1
scpa2.txt	Mutation Only	264.50	+4.96%	1
scpa2.txt	Both	264.50	+4.96%	1
scpa3.txt	Greedy	270.00	+16.38%	—
scpa3.txt	Crossover Only	241.75	+4.20%	1

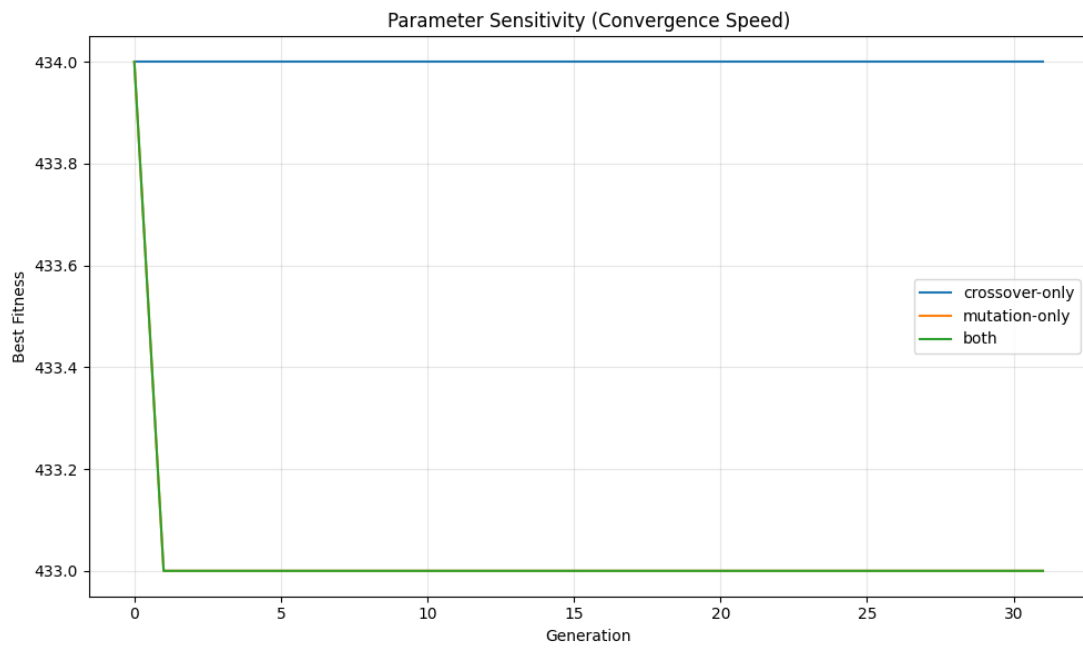
מערך נתונים	קונפיגורציה	עלות ממוצעת	פער מאופטימלי (%)	דורות ממוצעים
scpa3.txt	Mutation Only	242.00	+4.31%	1
scpa3.txt	Both	242.00	+4.31%	1

תוצאות עם סיד = 42 על כל המערכים

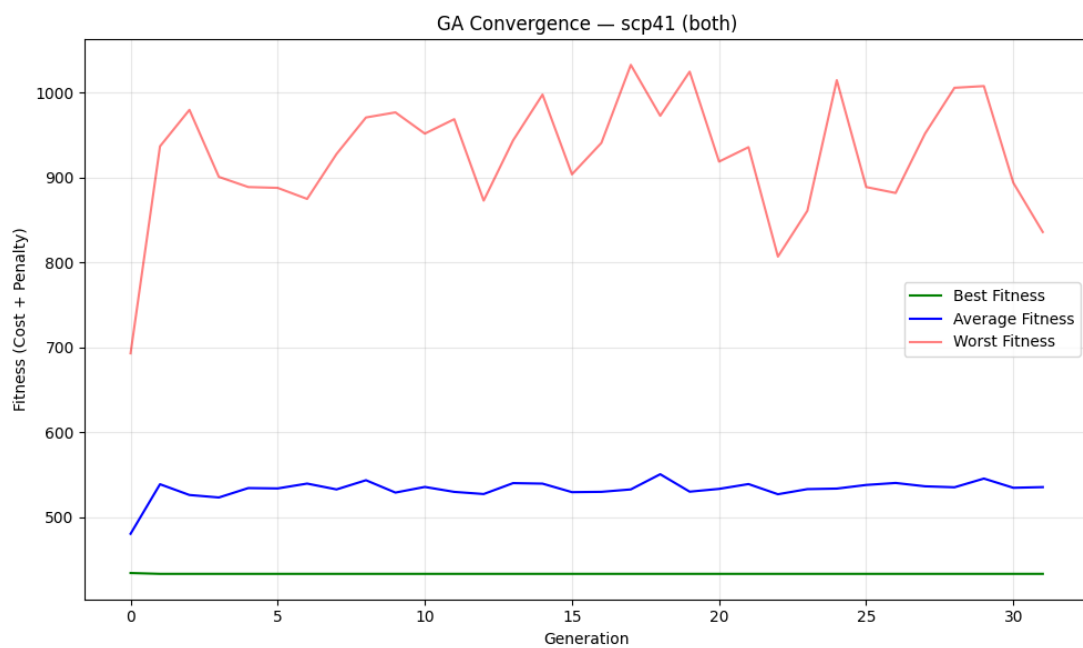
מערך נתונים	אופטימום	Greedy	פער גרידי	GA (seed=42)	פער GA
scp41	429	463	+7.93%	433	+0.93%
scp42	512	582	+13.67%	520	+1.56%
scp43	516	598	+15.89%	529	+2.52%
scp44	494	548	+10.93%	503	+1.82%
scp51	253	289	+14.23%	265	+4.74%
scp52	302	348	+15.23%	325	+7.62%
scp53	226	246	+8.85%	230	+1.77%
scpa1	253	288	+13.83%	259	+2.37%
scpa2	252	284	+12.70%	263	+4.37%
scpa3	232	270	+16.38%	241	+3.88%

14. גרפים:

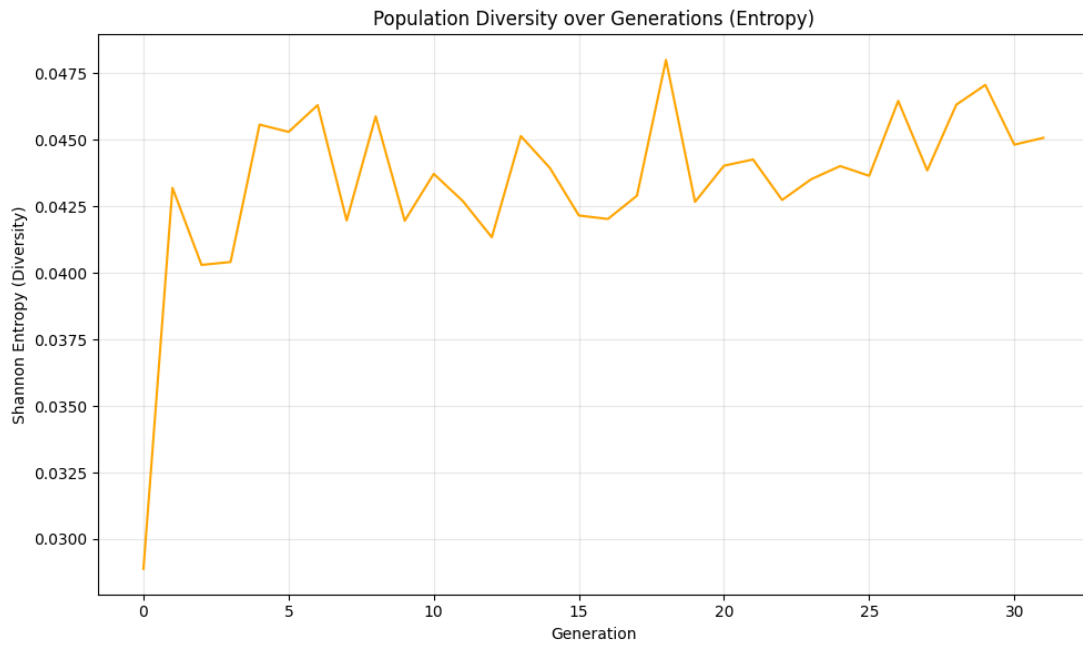
param_sensitivity_scp41_3configs.png קונפיגורציות (התכנסות)



Both קונפיגורציית convergence-scp41-both.png

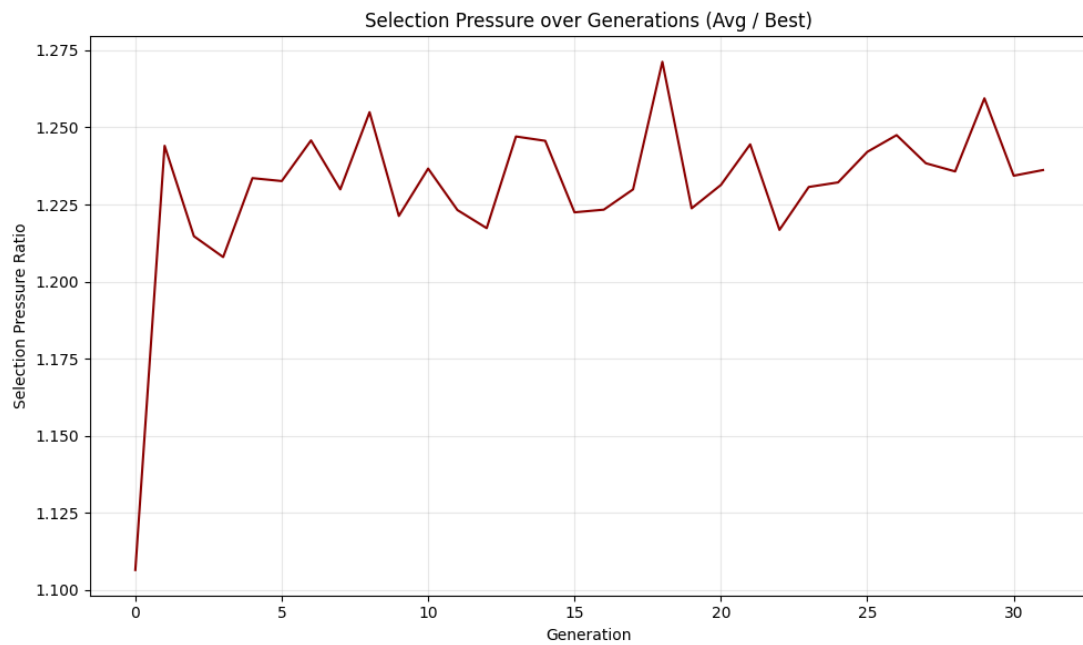


Both, אנטרופיה, diversity-entropy-scp41-both.png



.a

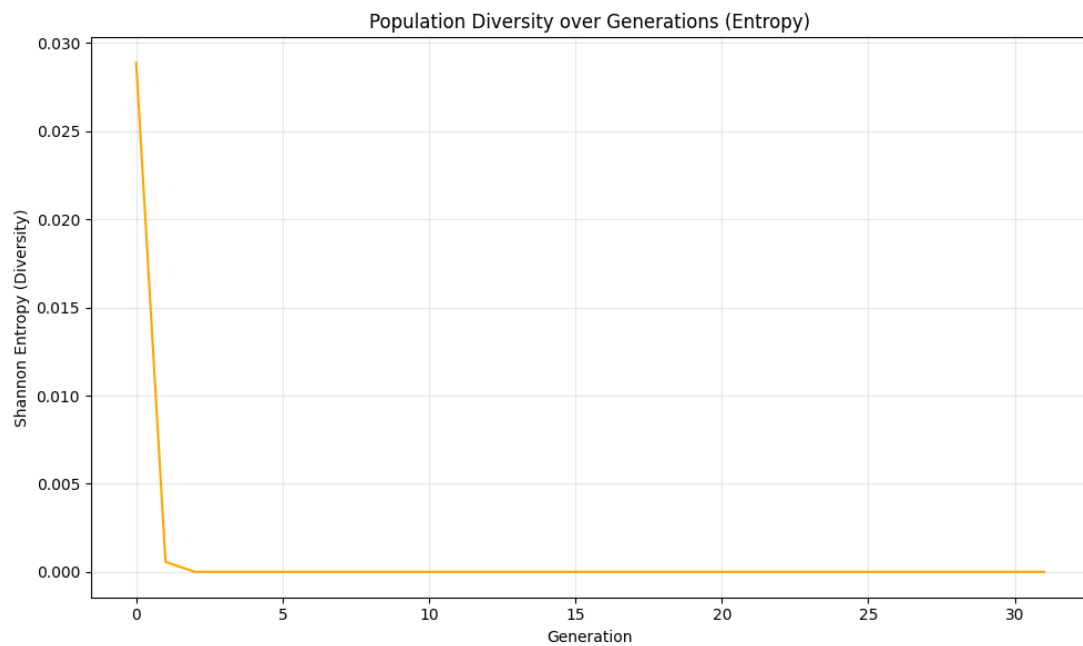
Both, לחץ סלקציה, selection-pressure-scp41-both.png



.b

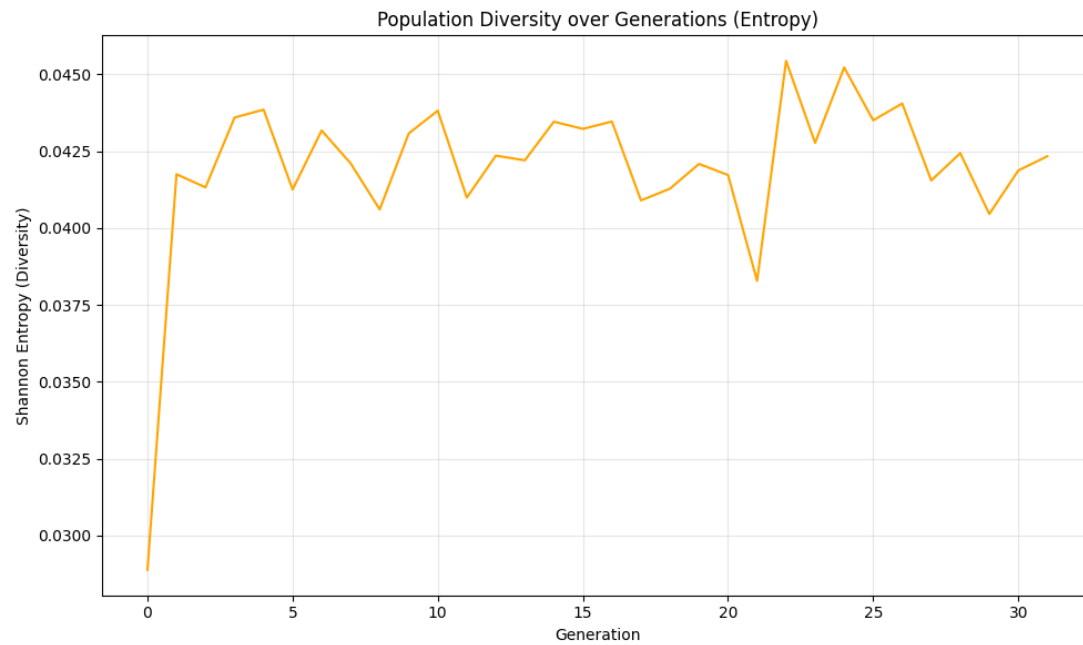
.c

Crossover Only, אנטרופיה, diversity-entropy-scp41-crossover-only.png



.d

Mutation Only, אנטרופיה, diversity-entropy-scp41-mutation-only.png



.e

15. תצפיות לגבי התכנסות: התצפית המרכזית בהרצת האלגוריתם עם הפרמטרים הרגילים (0.05) אפשר לשים לב שכל הריצות עוצרות כבר בדור 1 (!) בגלל התכנסות של האנטרופיה. האנטרופיה ירדה ל-0.04 כבר אחרי האתחול, לפני שהאלגוריתם הספיק לבצע סיבוב הכלאה אחד.

הסיבה: מנגנון התיקון היה אגרסיבי מאוד, ולאחר שמחילים תיקון + טרים על כל 300 הכרומוזומים הם מתכנסים לפתרונות דומים מאוד (אותן קבוצות טובות נבחרות בגרידי על ידי כולם). לכן מתקבל אנטרופיה שמודדת כמה מעורב הג'ן בכל מיקום, אז יש קריבה לפני שאפילו דור אחד של האבולוציה מתרחש.

לכן כדי לייצר גרפים משמעותיים שמראים אבולוציה אמיתית הריצה שונתה ל $entropy_threshold=0.0$ (כלומר לעצור את העצירה המוקדמת של התכנסות האנטרופיה). בתנאים האלו האלגוריתם ממשיך עד לעמידה בתנאי הסטגנציה של 30 דורות ללא שינוי.

ניתן ללמוד מכך שGA שלנו בשלב האתחול הוא בפועל חיפוש גרידי מקבילי, המיטוב האמיתי קורה רק בשלבים של האתחול + התיקון ולא בשלב האבולוציה. כדי לייצר אבולוציות משמעותיות יש לשקול או תיקון חלקי בלבד או הוספת רנדומליות לשלב התיקון. ואפשר כמ' לנסות להגדיל את הגיוון באתחול. אבל למעשה קיבלנו ביצועים סופיים טובים יותר מאשר גרידי בסיסי במקרים הללו.

מסקנות:

חלק א: השימוש ביורסטיקה $h1$ שהיא פשוטה יותר הוכיחה את עצמה כיעילה יותר והורידה את ממוצע הנודים לעומת הריצת BFS מ2478 ל2074. הEBF ירד מ1.635 ל1.399. בנוסף $h2$ שיפרה את המצב לעומת BFS, אבל לא היה משופר לעומת $h1$. בגלל עלות גבוהה יותר לנוד. עבור המערכת של המשחק הזה, A^* עם $h1$ הוא הבחירה הפרקטית שמוצא את כלל הפתרונות בצורה המיטבית.

חלק ב: מנגנון התיקון הוכיח את עצמו בתיקון בין הדורות, האוכלוסייה כולה תמיד חוקית ולכן אין צורך לכוון מקדמי ענישה. הGA הגיע לתוצאות בפער של 1-8% מהאופטימום לעומת 8-16% של הגרידיץ' הגילוי העיקרי היה שמנגנון תיקון אגרסיבי למעשה ממזג את האוכלוסייה כבר בדור 0 ומותיר מעט מקום לאבולוציה אמיתית.